Solution: Excel Sheet Column Title

Problem Statement

Given a positive integer `n`, return the string referring to the corresponding `column title` in the `Excel sheet`.

Excel titles are composed of uppercase letters, where "A" denotes the first column, "B" the second, up to "Z" for the 26th column. After "Z", the titles continue with two-letter combinations starting with "AA", "AB", and so on.

Examples

1. **Example 1**
   - **Input**: `700`
   - **Expected Output**: "ZX"
   - **Justification**: The number 700 corresponds to the column title "ZX" in Excel, where "Z" is the 26th letter, and "X" follows in the sequence, indicating 26*26 + 24.
2. **Example 2**
   - **Input**: `52`
   - **Expected Output**: "AZ"
   - **Justification**: The number 52 translates to "AZ", where "A" denotes the first cycle of 26, and "Z" is the 26th letter.
3. **Example 3**
   - **Input**: `705`
   - **Expected Output**: "AAC"
   - **Justification**: For the number 705, the column title is "AAC", which represents 26*26 + 1*26 + 3, indicating two cycles through the alphabet plus three letters into the next cycle.

Solution

To solve this problem, we adopt a method that iteratively determines each letter in the column title by dividing the input number by 26 (the number of letters in the English alphabet) and finding the remainder. This approach works because each letter in the title can be seen as a digit in a base-26 numeral system, where "A" represents 1, "B" represents 2, and so on, up to "Z" representing 26. However, since there is no "zero" in this system, we adjust by treating any remainder of 0 as 26 and decrementing the dividend accordingly before the next iteration. This method is effective because it directly maps the numerical value to its equivalent alphabetical representation in a straightforward, iterative manner, mimicking the process of converting numbers to a different base, which is exactly what the problem requires.

Step-by-step Algorithm

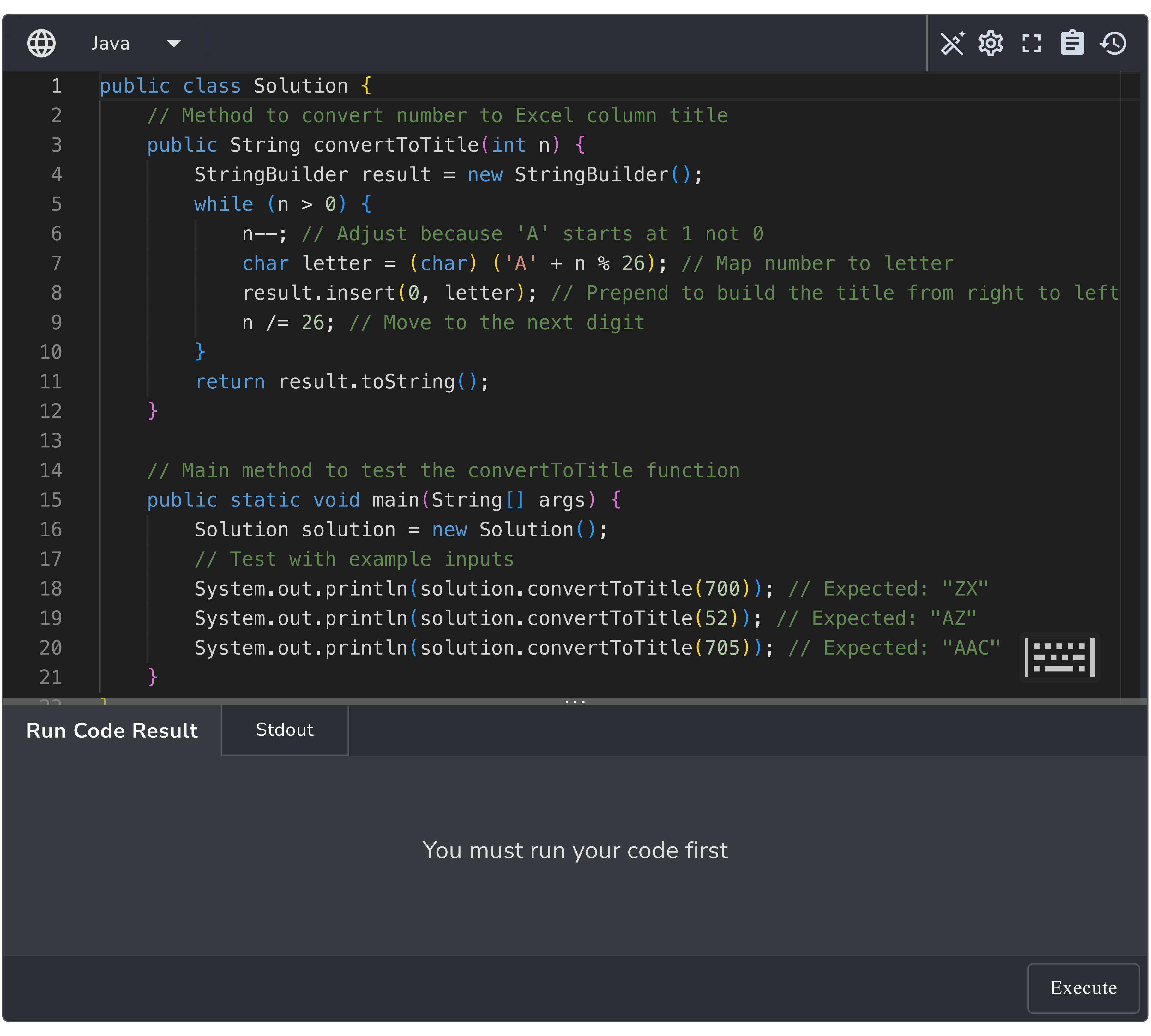
1. **Initialize an empty string** named `result` which will be used to build the column title in reverse order.
2. **Loop until the input number `n` is greater than 0.** This loop will repeatedly divide `n` by 26 to process each digit in the base-26 number system equivalent.
3. **Inside the loop, first decrement `n` by 1.** Excel's column numbering starts at 1 (not 0), so we adjust `n` to fit a 0-indexed system. This adjustment allows for a straightforward mapping of numbers to letters ('A' to 'Z').
4. **Calculate the current letter** by finding the remainder of `n % 26`. This step determines the current "digit" in the base-26 number. Then, map this number to its corresponding letter in the alphabet. Since 'A' corresponds to 0 after our adjustment, add the ASCII code of 'A' to the remainder to get the correct letter.
5. **Prepend this letter to `result`.** Since we are processing the digits in reverse order (from least significant to most significant), we add each new character to the beginning of `result`.
6. **Divide `n` by 26** to move to the next most significant digit.
7. **After exiting the loop, return `result`.** This string now contains the Excel column title corresponding to the original number.

Algorithm Walkthroug

Consider the input `705`:

1. **Initialization**: Start with `n = 705` and an empty string `result`.
2. **Iteration 1**:
   - Decrement `n`: `n = 705 - 1 = 704`.
   - Calculate `704 % 26 = 2`, which corresponds to the third letter 'C' (since 'A' = 0, 'B' = 1, 'C' = 2 in our adjusted system).
   - Prepend 'C' to `result`, so `result = "C"`.
   - Divide `n` by 26: `704 / 26 = 27`.
3. **Iteration 2**:
   - Decrement `n`: `n = 27 - 1 = 26`.
   - Calculate `26 % 26 = 0`, corresponding to 'A' (after adjustment).
   - Prepend 'A' to `result`, so `result = "AC"`.
   - Divide `n` by 26: `26 / 26 = 1`.
4. **Iteration 3**:
   - Decrement `n`: `n = 1 - 1 = 0`.
   - Since `n` is now 0, we would typically end, but the loop condition checks this at the start of each iteration. Before decrementing, `n = 1`, which corresponds to 'A'.
   - Prepend 'A' to `result`, making it "AAC".
   - The loop ends as `n` is now 0 after the division in the previous step.
5. **Result**: The final result string is "AAC", which is the correct Excel column title for the number `705`.

Code



```java
public class Solution {
    // Method to convert number to Excel column title
    public String convertToTitle(int n) {
        StringBuilder result = new StringBuilder();
        while (n > 0) {
            n--; // Adjust because 'A' starts at 1 not 0
            char letter = (char) ('A' + n % 26); // Map number to letter
            result.insert(0, letter); // Prepend to build the title from right to left
            n /= 26; // Move to the next digit
        }
        return result.toString();
    }

    // Main method to test the convertToTitle function
    public static void main(String[] args) {
        Solution solution = new Solution();
        // Test with example inputs
        System.out.println(solution.convertToTitle(700)); // Expected: "ZX"
        System.out.println(solution.convertToTitle(52)); // Expected: "AZ"
        System.out.println(solution.convertToTitle(705)); // Expected: "AAC"
    }
```

You must run your code first

Complexity Analysis

- **Time Complexity**: $O(logn)$, where n is the input number. This is because with each iteration, we divide the input number by 26, effectively reducing the problem size logarithmically.
- **Space Complexity**: $O(1)$, as the space used by the algorithm is constant, and it only requires a finite number of variables regardless of the input size.

← Previous
Next →
Excel Sheet Column Title (Easy)
Merge Strings Alternately (Easy)
☐ Mark as Completed